Chapter 21

Case Studies and Capstone Projects

21.1 Introduction

Every dataset used so far in this book, Cascade Home & Garden's satellite-derived car counts, Meridian Fabrication's financial statements, the ten-transaction fraud-scoring sample, was built for one purpose: to make a method's mechanics checkable by hand, free of the mess real data carries. That was the right choice for teaching each method in isolation, but it leaves a gap this closing chapter fills. A real financial dataset, of the kind a data scientist downloads from a public repository such as the UCI Machine Learning Repository or Kaggle, arrives with undocumented category codes, an imbalanced target, correlated and redundant features, and no guarantee that the model that wins on one metric also wins on every other metric that matters.

This chapter works one such dataset end to end, using only tools this book has already built: feature engineering and a model bake-off spanning linear and tree-based methods, from Chapter 6; Chapter 18's SHAP and permutation-importance methods, now run with the actual `shap` library rather than by hand; a cost-sensitive threshold choice extending Chapter 15's alert economics; and a fairness audit extending Chapter 18's synthetic two-group example to real data. One topic is new, calibration, motivated by Chapter 9's caveat that a credit score is meaningful only if the probability underneath it is accurate. The chapter closes with governance considerations and further capstone project ideas for the reader.

Two more advanced analyses go beyond a first pass through the pipeline: a systematic hyperparameter search measuring how much tuning gains over a single reasonable configuration, and an intersectional fairness cut that splits the test population two ways at once and uncovers a clear four-fifths-rule violation that the chapter's single-attribute audit, run first, missed.

The dataset is the *Default of Credit Card Clients* dataset (Yeh and Lien 2009): 30,000 Taiwanese credit card accounts with 23 raw features and a binary label, default on payment the following month, hosted on the UCI Machine Learning Repository and widely distributed on Kaggle under the same name. Unlike every other dataset in this book, it was not built to illustrate anything, and working through it surfaces problems no fictional dataset would: undocumented category codes that must be cleaned before any model is fit, a real 22.1% default rate rather than a convenient round number, and, as the fairness audits show, a model that passes a single-attribute fairness check comfortably while failing an intersectional one, although no protected attribute is among its inputs.

The chapter follows the order a data scientist follows on a new dataset. It begins by inspecting the raw data closely enough to find its undocumented quirks (Section 21.2.1) and engineering features from raw monthly statements rather than from a clean feature table (Section 21.2.2). It then compares several model families on the same held-out data (Section 21.3.1) and checks whether the winner's probabilities can be trusted at face value (Section 21.3.2). From there it explains that model's behavior with real software (Section 21.3.4), converts its output into a lending decision under an explicit cost assumption (Section 21.3.5), and audits that decision for fairness, first along one dimension (Section 21.3.6) and then along two at once (Section 21.4.4). A discussion of what deploying the pipeline would require (Section 21.4.1) is followed by a reusable template distilled from it (Section 21.4.5) and by further capstone projects to apply the template to (Section 21.4.6). Those end the chapter, and the book, on the note Chapter 1 opened with: these tools are only as trustworthy as the discipline applied in using them.

Wulin.Suo@Queensu.ca

21.2 The Data

The capstone begins where every real project does: with a dataset that is not what its documentation claims.

21.2.1 The Capstone Dataset and Its Imperfections

The raw dataset records, for each of 30,000 accounts, a credit limit (`LIMIT_BAL`), demographic fields (`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), six months of repayment status codes (`PAY_1` through `PAY_6`, where -1 indicates payment made duly and a positive integer indicates the number of months a payment is overdue), six months of billed amounts (`BILL_AMT1`–`BILL_AMT6`), six months of amounts actually paid (`PAY_AMT1`–`PAY_AMT6`), and the target, default on the following month's payment.

Loading the data surfaces the first imperfection the fictional datasets never had: the data dictionary documents `EDUCATION` as taking values 1 (graduate school), 2 (university), 3 (high school), or 4 (other), yet the column also contains 14 rows coded 0, 280 rows coded 5, and 51 rows coded 6, none of which the documentation defines. `MARRIAGE` shows the same pattern: documented as 1 (married), 2 (single), or 3 (other), it also contains 54 rows coded 0.

Neither anomaly is disclosed in the documentation, and a modeler who does not notice them will silently feed a model category codes with no defined meaning. Following standard practice for this well-studied dataset, both sets of undocumented codes are folded into their "other" categories (0, 5, and 6 into `EDUCATION`'s category 4; 0 into `MARRIAGE`'s category 3) before any modeling, a cleaning step with no equivalent elsewhere in this book because every other dataset was built clean.

The repayment-status columns carry two further quirks of the same kind. First, the raw file has no `PAY_1` at all: the most recent month's status column is named `PAY_0`, while the billed and paid amounts run from `BILL_AMT1` and `PAY_AMT1`, so the companion notebook renames `PAY_0` to `PAY_1` to line the three six-month series up. Second, the dictionary defines the status codes as -1 (paid duly) and 1 through 9 (months overdue) only, yet the columns are dominated by two undocumented codes, -2 and 0, which together make up 66.9% of all monthly status entries and appear at least once for 25,939 of the 30,000 accounts. They are commonly read as "no balance to pay" and "revolving credit, minimum payment made," but nothing in the official documentation says so. This chapter leaves them as they are; since both rank below 1, neither counts as a delinquency in the `max_delinquency` feature defined below, which is a modeling decision a reader should notice rather than inherit silently.

Table 21.1 summarizes four representative raw fields before any cleaning or feature engineering; it is worth inspecting closely, the habit Chapter 6 recommended before fitting any model.

Field	Mean	Std. dev.	Min	Median	Max
`LIMIT_BAL` (NT$)	167,484	129,748	10,000	140,000	1,000,000
`AGE` (years)	35.5	9.2	21	34	79
`BILL_AMT1` (NT$)	51,223	73,636	−165,580	22,382	964,511
`PAY_AMT1` (NT$)	5,664	16,563	0	2,100	873,552

Table 21.1: Summary statistics for four raw fields (before cleaning)

`BILL_AMT1`'s negative minimum is another undocumented quirk: a negative billed amount means the account was in credit, having overpaid the previous month, a legitimate state for a revolving credit account but one that a modeler unfamiliar with the data might mistake for a data-entry error and drop or clip without justification. `PAY_AMT1`'s heavy right skew, a median of just NT$2,100 against a mean of NT$5,664 and a maximum of NT$873,552, is the kind of distribution Section 21.2.2's ratio features are designed to normalize before they reach a linear model, though the tree-based models of Section 21.3.1 are much less sensitive to skew than logistic regression, since a tree splits on a feature's rank order rather than its raw scale.

Two more properties of the raw data matter before any model is fit. The first is the class balance. A 22.1% default rate is much higher than a typical consumer credit-card portfolio, which more commonly runs in the low single digits: a published research dataset's class balance reflects how and when it was sampled, not the balance another institution's portfolio will show. This data covers a six-month window in 2005, shortly before a Taiwanese consumer-credit crisis, so a model built on it should not be assumed to transfer

Wulin.Suo@Queensu.ca

its absolute predicted probabilities to a different portfolio without recalibration, even if its ranking of riskier against safer accounts transfers reasonably well.

The second property is redundancy. Several raw features are highly correlated by construction: the six `BILL_AMT` columns track one account's revolving balance from month to month, so they move together. That matters for Section 21.3.4's explanations, because an importance score assigned to any one of several correlated features is partly arbitrary when the model can substitute one for another with little effect on its predictions. Chapter 6 raised this issue as multicollinearity in linear regression, and it carries over, in a different technical form, to tree-based models and their explanations.

21.2.2 From Raw Fields to Model-Ready Features

Chapter 6 introduced feature engineering in the abstract; this section performs it on raw fields for the first time. Five engineered features summarize six months of repayment and billing history in a form a model can use more directly than six separate monthly columns. `max_delinquency` is the worst (maximum) repayment-status code across the six months, capturing whether an account was ever seriously overdue even if its latest month looks clean. `avg_utilization` is the six-month average of billed amount divided by credit limit, a standard credit-risk signal Chapter 9 introduced but never computed from raw statements. `total_bill` and `total_paid` are the six-month sums of billed and paid amounts, and `pay_to_bill_ratio` is their ratio, capturing how much of what an account owed it paid back over the window; it is clipped at 5 to bound the influence of a few accounts with a large payment relative to a near-zero bill. These five, together with 20 of the 23 raw fields, give 25 features. The three raw fields left out are `SEX`, `MARRIAGE`, and `AGE`: taking U.S. fair-lending law as the reference point, as Chapter 18 did, the Equal Credit Opportunity Act and its Regulation B forbid basing a credit decision on sex or marital status and allow age only in narrow ways (for instance, as a variable in a demonstrably and statistically sound scoring system that never treats applicants aged 62 or older less favorably), so the cleanest choice is to keep all three out of the model and hold `SEX` and `AGE` aside as attributes for the fairness audits of Sections 21.3.6 and 21.4.4.

The 30,000 accounts are split 80/20 into a training set of 24,000 and a held-out test set of 6,000, stratified by the default label so both sets keep the 22.1% default rate, with a fixed random seed so that the split, and every result in this chapter, is reproducible from the companion notebook. Every model comparison, calibration check, explanation, threshold choice, and fairness audit below is computed on this held-out test set, never on data a model was fit to. Chapter 6 introduced that discipline as essential to detecting overfitting, and it matters more once a real dataset's quirks make it easy to believe a model has learned something it has only memorized.

A real dataset also makes another discipline easy to violate: every engineered feature, `max_delinquency` included, is computed from information available when each account's repayment history was recorded, not from anything that depends on knowing whether the account defaulted. Chapter 17's point-in-time discipline applies here in a subtler form: a feature engineered carelessly from a dataset that also contains the outcome, for instance a rolling average that accidentally includes a future month's billing data, can leak the label into the features and produce a model that looks excellent on a test set drawn from the same leaking dataset yet fails once deployed on new applicants for whom the leak is unavailable. This dataset's structure makes such leakage easy to avoid, since every raw field precedes the single target month, but the habit of checking for it explicitly, rather than assuming a clean-looking dataset has none, is one to carry into Section 21.4.6's further projects, several of which (Lending Club in particular) are much easier to leak by accident.

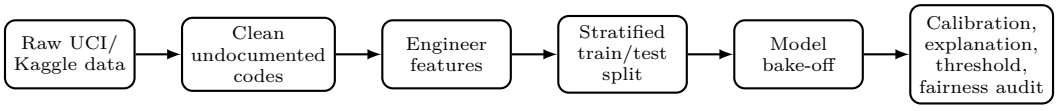


Figure 21.1: The end-to-end pipeline this chapter works through by hand

Wulin.Suo@Queensu.ca

21.3 Modeling and Evaluation

This part runs a full bake-off, then asks the questions that decide whether any of it is usable.

21.3.1 A Model Bake-Off: Logistic Regression, Random Forests, and Gradient Boosting

Four models are fit on the training set and evaluated, once, on the untouched test set: a plain logistic regression (Chapter 6's workhorse, standardized features, no adjustment for class imbalance), a class-weighted logistic regression (Chapter 15's first remedy for imbalanced classification, applied here directly rather than to a stylized example), a random forest (300 trees, Chapter 6's bagging-based ensemble), and gradient-boosted trees via XGBoost (300 boosting rounds, Chapter 6's sequential, bias-reducing ensemble family, named but not fit by hand in earlier chapters).

Table 21.2 reports seven metrics for each.

Model	Accuracy	Precision	Recall	F_1	ROC-AUC	PR-AUC	Brier
Logistic (plain)	0.807	0.683	0.238	0.353	0.720	0.494	0.146
Logistic (balanced)	0.731	0.426	0.615	0.503	0.721	0.485	0.205
Random Forest	0.816	0.660	0.350	0.458	0.775	0.554	0.136
XGBoost	0.818	0.665	0.356	0.464	0.778	0.557	0.135

Table 21.2: Held-out test set performance for four models on the credit-default dataset

The plain logistic regression's accuracy (80.7%) looks respectable in isolation, the trap Chapter 15 warned against for a rare-event problem: with a 22.1% default rate, a model far too conservative about flagging defaults can still post high accuracy by predicting the majority class, and this model's recall of only 23.8% shows it misses more than three-quarters of the accounts that default at the standard 0.5 threshold. Class weighting flips the trade-off as Chapter 15 described: recall rises to 61.5% while precision falls from 68.3% to 42.6%, a much better F_1 (0.503 versus 0.353) but a distinctly worse Brier score (0.205 versus 0.146). That pairing is the book's first concrete evidence that class weighting is not a free improvement: reweighting the training loss to counter imbalance also pulls the model's output away from a well-calibrated probability, a cost Chapter 15 never quantified with a metric like the Brier score, since its worked example used a balanced confusion-matrix illustration rather than a held-out calibration check.

The two tree ensembles dominate both discrimination metrics, ROC-AUC and PR-AUC, over either logistic regression. The comparison also gives Chapter 6's bagging-versus-boosting distinction its first empirical instance: XGBoost's sequential, error-correcting boosting edges out the random forest's parallel, variance-reducing bagging on every metric in the table, though by a narrower margin (ROC-AUC 0.778 versus 0.775) than the gap either ensemble opens over logistic regression. That pattern matches a finding well documented in applied machine learning: gradient boosting and random forests are far closer competitors with each other on tabular data than either is with a linear model, though boosting often comes out slightly ahead.

The comparison also corrects a common oversimplification, that a logistic regression is always the "safe, interpretable" choice and a tree ensemble the "powerful, opaque" one. Both ensembles beat both logistic regressions on every ranking-based metric by roughly five to six points of ROC-AUC while remaining explainable in Chapter 18's sense, with the tooling Section 21.3.4 applies below, though not in the coefficient-reading sense that made Chapter 9's linear credit-scoring model interpretable by construction.

A lender choosing among these four models is therefore not trading accuracy for interpretability along a single dial; it is trading a small amount of post-hoc explanation overhead (Section 21.3.4's SHAP computation) for a measurable gain in discrimination and, as Section 21.3.2 shows next, calibration.

Wulin.Suo@Queensu.ca

21.3.2 Is the Model's Probability Actually a Probability? Calibration in Practice

Chapter 9 noted that a credit score is meaningful only to the extent that the default probability underneath it is well calibrated, not merely useful for ranking applicants from safest to riskiest; this section makes that check concrete.

Calibration asks whether, among all accounts a model assigns a predicted default probability of, say, 30%, roughly 30% actually default. Table 21.3 bins the test set into ten deciles of predicted probability for the plain logistic regression and for XGBoost, and compares each decile's average predicted probability to its actual observed default rate.

	Logistic (plain)		XGBoost	
Decile (low to high)	Mean predicted	Mean actual	Mean predicted	Mean actual
1	0.040	0.135	0.036	0.045
5	0.167	0.118	0.131	0.152
6	0.185	0.120	0.159	0.183
10	0.577	0.670	0.725	0.697

Table 21.3: Calibration by predicted-probability decile: plain logistic regression versus XGBoost

XGBoost's predicted and actual rates stay within three percentage points of each other in every decile shown (and, in the companion notebook's full table, in every decile not shown), consistent with its lower Brier score in Table 21.2. The plain logistic regression shows a visible gap in its lowest decile, an average predicted probability of 4.0% against an actual default rate of 13.5%, more than three times higher: the safest-looking accounts by this model's own ranking are much riskier than its probabilities suggest, a miscalibration a lender relying on the raw probability, not just the ranking, would want caught before deployment.

Figure 21.2 draws all ten deciles of both models as a **reliability diagram**, together with the isotonic recalibration of the next subsection. Unlike every other figure in this book except those of Chapters 2 and 5, it and the two that follow are the companion notebook's own matplotlib output rather than redrawn diagrams, because on real data the scatter of the points is itself the evidence. A perfectly calibrated model lies on the diagonal. XGBoost's points hug it from the safest decile to the riskiest. The plain logistic regression's zigzag across it at the low end, where its four lowest deciles predict between 4.0% and 14.2% while the observed default rates jump between 8.7% and 16.8%, sag below it through the middle deciles, and finish well above it in the top decile, 57.7% predicted against 67.0% observed.

This is why Section 21.3.5's cost-sensitive threshold and Section 21.3.6's fairness audit both use XGBoost's probabilities rather than the plain logistic regression's: both depend on the predicted probability meaning what it claims, not merely on the model's ability to rank risky accounts above safe ones.

21.3.3 Fixing Miscalibration: Isotonic Recalibration

The plain logistic regression's calibration gap raises the next question: can it be corrected without discarding the model? **Isotonic recalibration** fits a separate, nonparametric, monotonically increasing function that remaps a model's raw probabilities onto better-calibrated ones. The remapping is learned from the data the original model was fit on, but evaluated out-of-fold, Section 21.2.2's train/test discipline applied one level deeper, to the recalibration step itself. Fitting the remapping directly on the test set, as a careless analyst might, would leak the very data the recalibration is meant to be validated against.

Applied to the plain logistic regression, scikit-learn's `CalibratedClassifierCV` with isotonic regression, five-fold cross-validated entirely within the training set, improves the test-set Brier score from 0.1460 to 0.1419, a modest but real reduction, while leaving ROC-AUC essentially unchanged (0.720 before, 0.724 after). That is what an isotonic remapping should do: because the transformation is monotonic, it cannot change how the model ranks any two applicants, only how well its stated probability matches reality at a given rank. The improvement is concentrated where Table 21.3 found the problem.

Wulin.Suo@Queensu.ca

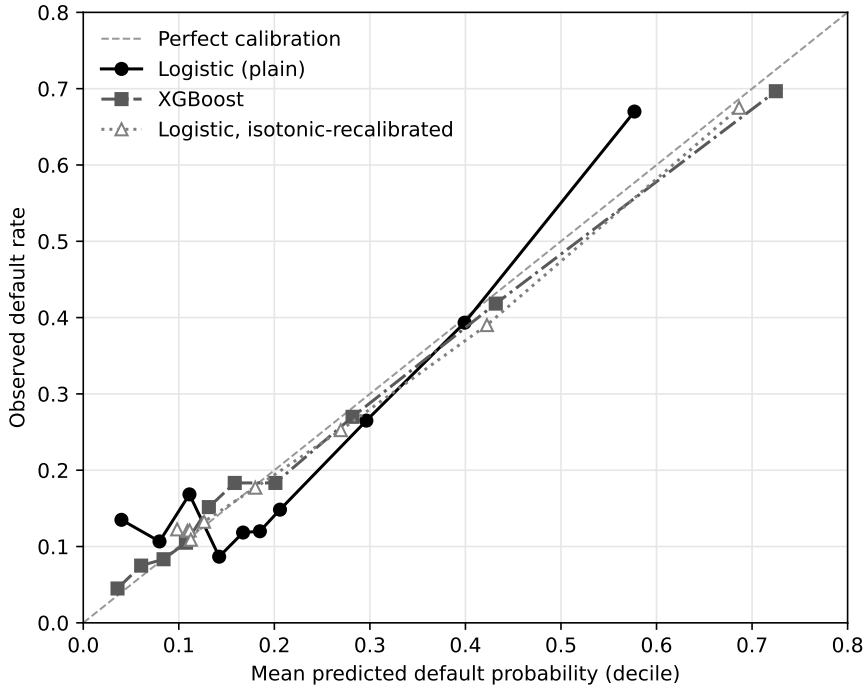


Figure 21.2: Reliability diagram on the 6,000-account test set: mean predicted default probability against observed default rate, by decile of each model's own predictions. The recalibrated logistic regression has nine bins because many of its scores tie.

Recomputing the reliability table on the recalibrated probabilities for the lowest-probability bin,

$$\begin{aligned} \text{Recalibrated:} &\quad \text{predicted} = 9.8\%,\ \text{actual} = 12.3\%,\ \text{ratio} \approx 1.2, \\ \text{Original (uncalibrated):} &\quad \text{predicted} = 4.0\%,\ \text{actual} = 13.5\%,\ \text{ratio} > 3, \end{aligned}$$

The lesson is general: a model with good discriminative power but poor calibration, the plain logistic regression's situation here, need not be replaced to become a trustworthy probability estimate, only recalibrated, a much cheaper fix than retraining a different model family, and one that leaves all the original coefficients, and so the model's adverse-action interpretability, intact.

Recalibration does not fix everything: XGBoost's Brier score (0.135) remains lower than the recalibrated logistic regression's (0.142), since XGBoost's advantage was never only a calibration artifact. Its higher ROC-AUC in Table 21.2 reflects superior discriminative power that no monotonic remapping can extract from a linear model's more limited functional form.

Calibration matters beyond this chapter's threshold and fairness exercises. A bank reporting expected credit losses under IFRS 9 or the analogous U.S. CECL framework multiplies a probability of default directly into a provisioning calculation, Chapter 9's EL = PD × LGD × EAD decomposition. A systematically miscalibrated probability, not merely a poorly ranked one, therefore becomes an over- or understated loss provision in the bank's financial statements, the kind of number Chapter 4 taught how to read and Chapter 9 how to model, and one that depends on a property, calibration, that neither accuracy nor ROC-AUC guarantees.

Section 21.3.4 returns to this formula with real provisioning numbers, once this chapter's own low-risk and high-risk applicants have been introduced.

Wulin.Suo@Queensu.ca

21.3.4 Explaining a Real, Nonlinear Model: SHAP and Permutation Importance

Chapter 18 computed an exact Shapley decomposition by hand for Chapter 9's credit-scoring model only because that model was linear in the log-odds; XGBoost is not. The `shap` library's `TreeExplainer` algorithm computes exact Shapley values efficiently for tree ensembles by exploiting the tree structure directly, rather than relying on the linear shortcut or on sampled perturbations as Chapter 18's LIME example did.

Table 21.4 compares the resulting global feature ranking, mean absolute SHAP value across 1,000 held-out accounts, against Chapter 18's other global method, permutation importance (here, the drop in ROC-AUC when a feature is shuffled), computed on the full test set.

Rank	Permutation importance (feature)	Mean |SHAP| (feature)
1	`PAY_1` (0.0337)	`max_delinquency` (0.444)
2	`max_delinquency` (0.0318)	`PAY_1` (0.339)
3	`avg_utilization` (0.0118)	`avg_utilization` (0.167)
4	`BILL_AMT1` (0.0094)	`BILL_AMT1` (0.130)
5	`LIMIT_BAL` (0.0057)	`LIMIT_BAL` (0.118)

Table 21.4: Top features by two global importance methods, XGBoost model

The two methods agree on which five features matter (the most recent repayment status, the worst repayment status over six months, utilization, the most recent bill, and credit limit) and order ranks three through five identically, but disagree on the top two: permutation importance ranks the most recent month's repayment status (`PAY_1`) above the worst status across six months, while SHAP ranks them the other way around. Neither ranking is wrong; they answer different questions, as Chapter 18 cautioned: permutation importance describes a feature's contribution to a specific performance metric on specific data, while SHAP describes each feature's average marginal contribution to individual predictions. A modeler who reports only one of the two risks a false precision about which single feature "matters most."

A single applicant's explanation shows at the individual level what Section 21.3.1's aggregate table cannot. The first test-set applicant is assigned a predicted default probability of 13.07% by XGBoost.

SHAP's additive decomposition operates on the model's raw log-odds output rather than on the probability, as Chapter 18's exact Shapley decomposition did for the linear credit-scoring model. This applicant's base value, the average log-odds prediction across the training population, is -1.272, and summing the SHAP contributions of every feature gives a total log-odds of -1.895. The sigmoid converts that back to 13.07%, matching the model's output: the completeness property Chapter 18 checked by hand for a linear model, now confirmed for a nonlinear ensemble. The applicant's largest contributions are

$$\begin{aligned} \texttt{max_delinquency} = 0 \text{ (no serious delinquency):} &\quad \text{SHAP} = -0.309, \\ \texttt{avg_utilization} = 12.1\% \text{ (low):} &\quad \text{SHAP} = -0.215, \\ \text{Credit limit} = \text{NT\$50,000 (modest):} &\quad \text{SHAP} = +0.175, \\ \text{Most-recent repayment status} = -1 \text{ (paid duly):} &\quad \text{SHAP} = -0.151, \end{aligned}$$

the modest credit limit contributing positively to predicted risk since, in this dataset, lower limits are associated with higher-risk accounts.

Figure 21.3 draws the whole decomposition as a **waterfall**: starting from the base value of -1.272, each bar moves the running log-odds by one feature's SHAP value, hatched bars raising predicted risk and plain bars lowering it. The seven largest contributions are shown individually; the remaining 18 features together contribute -0.247, and the bars end at -1.895, the model's actual output, which is the completeness property made visible.

Chapter 18's adverse-action reason codes followed automatically from a linear model's coefficients; for a deployed ensemble, producing the same kind of individualized explanation requires this extra step, running `TreeExplainer` against the fitted model, before a bank can issue a model-derived reason to an applicant under the Equal Credit Opportunity Act.

A contrasting high-risk applicant from the same test sample makes the method's value clearer. This applicant is assigned a predicted default probability of 92.2%, and the same decomposition gives a base value

Wulin.Suo@Queensu.ca

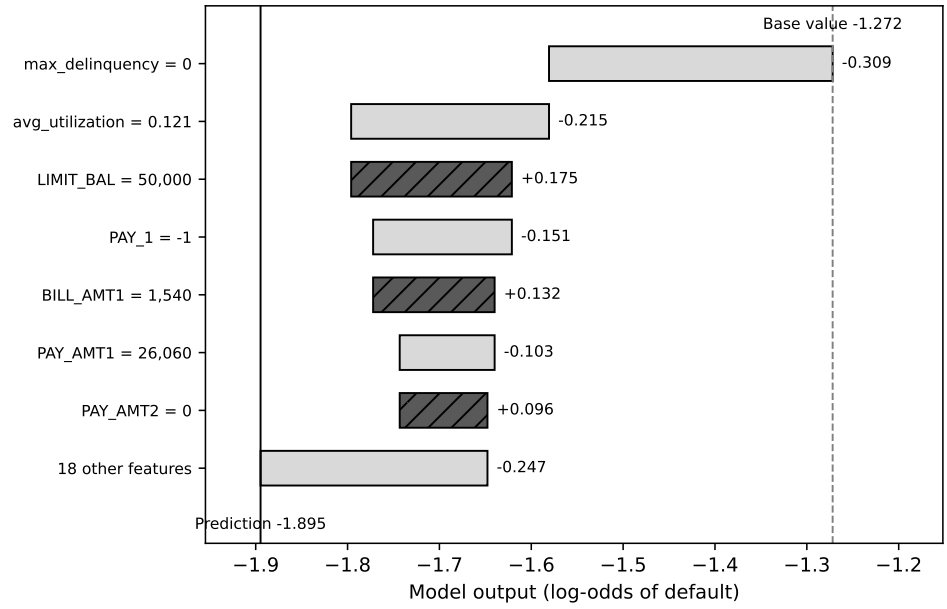


Figure 21.3: SHAP waterfall for the low-risk applicant (predicted default probability 13.07%), in log-odds. Hatched bars increase predicted risk; plain bars decrease it. The dashed line marks the base value and the solid line the model's output.

of −1.272 and a summed log-odds of 2.470 (whose sigmoid is, again exactly, 92.2%). Here the dominant contributions run the other way:

$$\begin{aligned} \text{Most-recent repayment status} = 3 \text{ (three months overdue):} &\quad \text{SHAP} = +1.009, \\ \text{Worst-ever repayment status} = 7 \text{ (severely delinquent):} &\quad \text{SHAP} = +0.921, \\ \text{Credit limit} = \text{NT\$10,000 (the dataset's minimum):} &\quad \text{SHAP} = +0.303, \end{aligned}$$

with `pay_to_bill_ratio` = 0 (repaid essentially nothing over six months) adding a further +0.151, and the most-recent-repayment contribution the single largest seen in either applicant's decomposition.

Side by side, the two explanations read almost as mirror images: no delinquency history and full repayment push one applicant's risk sharply down, while recent, severe delinquency and near-zero repayment push the other's sharply up. That individualized contrast is what an adverse-action notice or a credit-committee review needs, and what an aggregate importance ranking like Table 21.4 cannot provide.

Worked Example 21.1: From Predicted Probability to a Provisioning Number

Section 21.3.2 promised to return to Chapter 9's EL = PD × LGD × EAD decomposition with real numbers; the two applicants supply them. Each applicant's credit limit stands in for exposure at default, a standard simplification for a revolving line assumed fully drawn at default, and, with no collateral information for an unsecured card balance, the example reuses Chapter 9's illustrative LGD = 60%.

All amounts in this dataset are in New Taiwan dollars (NT$), not U.S. dollars; at the 2005 exchange rate of roughly NT$32 to the U.S. dollar, the low-risk applicant's NT$50,000 limit is worth only about US$1,550. For the low-risk applicant, PD = 13.07077% (the 13.07% reported earlier, carried to more decimals to reproduce the figure below exactly by hand) and EAD = NT$50,000 (this applicant's `LIMIT_BAL`), giving

$$\text{EL}_{\text{low-risk}} = 0.1307077 \times 0.60 \times \text{NT\$50,000} = \text{NT\$3,921.23},$$

Wulin.Suo@Queensu.ca

while the high-risk applicant, with PD = 92.19913% (similarly carried to more decimals) and a much smaller EAD = NT$10,000 (a lower credit limit, itself consistent with this applicant's weaker profile), gives

$$\text{EL}_{\text{high-risk}} = 0.9219913 \times 0.60 \times \text{NT\$10,000} = \text{NT\$5,531.95}.$$

The high-risk applicant's expected loss is only about 1.41× the low-risk applicant's, a far smaller multiple than the 7.05× gap in predicted default probability (92.20%/13.07%), because the high-risk applicant's credit limit, one-fifth of the other's, largely offsets the higher probability: an interaction among the three inputs to expected loss that looking at PD alone would miss. A bank provisioning against its whole card portfolio sums a version of this calculation across every account, so a systematic miscalibration of the kind Section 21.3.2 found in the plain logistic regression, understating risk in the lowest-probability decile, would understate provisions for the bank's safest-looking accounts, the segment a supervisor examining provisioning adequacy would be least likely to scrutinize.

`TreeExplainer` is also fast: it computes exact Shapley values for a tree ensemble in time roughly proportional to the number of trees times the square of tree depth, rather than the exponential cost of the general, model-agnostic Shapley formula Chapter 18 mentioned, which is why it can run here against a several-hundred-tree ensemble rather than only against the small linear toy models of Chapter 18's hand computations. This matters operationally: a bank issuing thousands of adverse-action notices a month needs an explanation method fast enough for that volume, and a sampling-based method like LIME, useful for a single illustrative explanation, does not scale as cleanly as an algorithm built for the model family being explained.

21.3.5 Choosing an Operating Threshold: A Real Cost-Sensitive Analysis

Every result so far used the default 0.5 threshold. Chapter 15's alert-economics section argued that the right threshold is a business decision balancing the cost of a missed default against the cost of denying a good customer, not a property of the model, and this section quantifies that trade-off. Assigning an illustrative cost of $500 to a missed default (a false negative, an account that defaults but is approved) and $50 to wrongly denying a good customer (a false positive, a tenth of the missed-default cost, since a denied good customer typically costs foregone interest rather than lost principal), the expected total cost across the test set at threshold t is $\text{Cost}(t) = 500 \cdot FN(t) + 50 \cdot FP(t)$.

Table 21.5 sweeps this across a grid of thresholds using XGBoost's predicted probabilities.

Threshold	False negatives	False positives	Total cost
0.05	29	4,071	$218,050
0.10 (cost-minimizing)	129	2,905	$209,750
0.20	450	1,218	$285,900
0.35	667	510	$359,000
0.50 (naive default)	854	238	$438,900
0.60	918	179	$467,950

Table 21.5: Cost-sensitive threshold sweep, XGBoost model, held-out test set

The cost-minimizing threshold on this grid is $t = 0.10$, far below the naive 0.5 default, at a total expected cost of $209,750, versus $438,900 at the naive threshold, more than double. That value could have been predicted without a sweep. Denying an account with predicted default probability p is cheaper than approving it whenever $p \times \$500 > (1 - p) \times \$50$, so for a calibrated model the cost-minimizing threshold is

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}} = \frac{50}{50 + 500} = 0.091, \tag{21.1}$$

the same expected-cost comparison Chapter 15 used to derive its fraud-review threshold, adapted to a decision in which a correctly denied defaulter costs nothing. Only the ratio of the two costs matters, not their level or currency. The grid's empirical minimum at 0.10 sits right next to this analytic value (at exactly 0.091

Wulin.Suo@Queensu.ca

the test-set cost is $214,100, the small difference being sampling noise in 6,000 accounts), a further check on Section 21.3.2's finding that XGBoost's probabilities are well calibrated.

Figure 21.4 plots the cost at every threshold on the 0.01-spaced grid behind Table 21.5. Its shape carries a practical message the table only hints at: the curve is nearly flat around its minimum, from $218,050 at $t = 0.05$ to $209,750 at $t = 0.10$, and then climbs steeply and almost linearly toward $438,900 at the naive threshold. Setting the threshold somewhat too low is cheap here; setting it far too high is expensive.

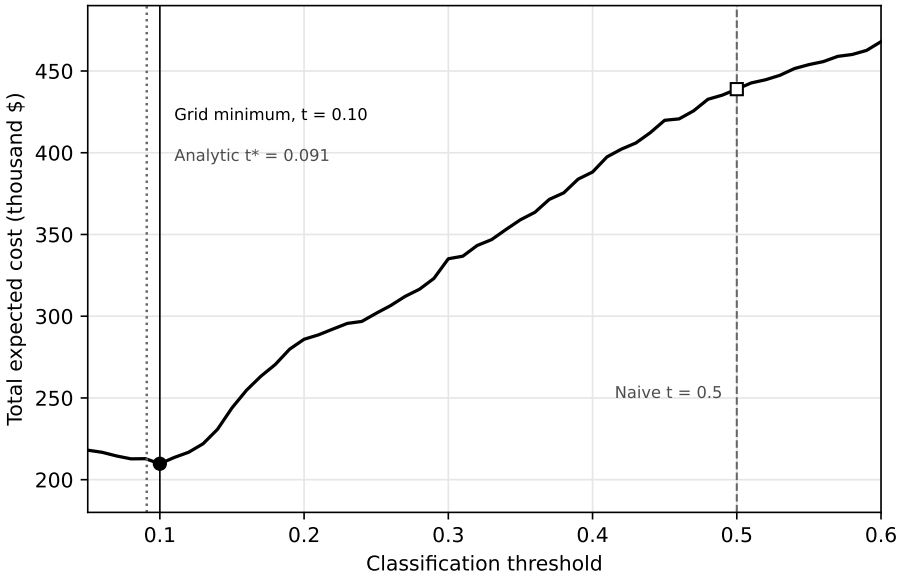


Figure 21.4: Total expected cost on the test set, $500\,FN(t) + 50\,FP(t)$, against the classification threshold t, XGBoost model. The solid vertical line marks the grid minimum at $t = 0.10$, the dotted line the analytic threshold $t^* = 0.091$, and the dashed line the naive $t = 0.5$.

The gap between $209,750 and $438,900 is the lesson: choosing a threshold by convention rather than by the institution's cost structure leaves real money on the table, much more here than the gap between any two models in Table 21.2, which is why Chapter 15 treated threshold choice as at least as consequential as model choice. The optimal threshold depends entirely on the assumed 10-to-1 cost ratio; a lender who judged missed defaults and denied good customers comparably costly would land much closer to 0.5. This is a business judgment, not a number the chapter's methods can supply.

The resulting operating point deserves a plain statement: at $t = 0.10$ the model approves only 31.6% of the test accounts, against 88.2% at the naive 0.5 threshold, and turns away 62.2% of the accounts that would not have defaulted. No real card lender operates anywhere near so strict a cutoff. The toy cost ratio drives it: charging a missed default only ten times a wrongly denied good customer, against a 22.1% base default rate, pushes the threshold below the probability most accounts are assigned. A real lender's cost of denying a good customer includes the lifetime value of the relationship, and the cost of a missed default is proportional to the exposure rather than flat, so a realistic cost model would place the threshold, and the approval rate, very differently.

The threshold sweep is a cost-weighted walk along the receiver operating characteristic curve Chapter 6 introduced. Each threshold in Table 21.5 is one point on that curve, trading fewer false negatives for more false positives as the threshold falls. What this section adds is not a new statistical concept but a conversion of that trade-off into dollars, so the operating point is chosen by comparing two cost figures rather than by picking a point on a curve that looks like a reasonable compromise.

A lender would also stress-test this choice, as Chapter 9 stress-tested a credit portfolio: if a recession doubled the cost of a missed default relative to a denied applicant, the cost-minimizing threshold would fall

Wulin.Suo@Queensu.ca

further, flagging more accounts as high-risk, the scenario analysis a model-risk function should run before committing to a single operating threshold in production.

21.3.6 A Real Fairness Audit

Chapter 18's fairness audit used a synthetic eight-applicant dataset built to make a point cleanly; this section runs the same audit, the four-fifths rule and an error-rate comparison, on the real test set at the cost-minimizing threshold of Section 21.3.5, split by the dataset's `SEX` field. Table 21.6 reports the results.

Group	n	Approval rate	False-negative rate	False-positive rate
Male	2,402	30.9%	8.9%	62.5%
Female	3,598	32.1%	10.3%	62.0%

Table 21.6: A fairness audit by sex, XGBoost model at the cost-minimizing threshold ($t = 0.10$)

The adverse impact ratio, computed from the unrounded approval rates as $30.85\%/32.13\% = 0.960$, clears the 80% four-fifths threshold of Chapter 18's fairness section by a wide margin, and some gap is expected, since male accounts in the test set default more often (23.4% against 21.3%). The error rates show the tension Chapter 18's synthetic example illustrated, now on real data: the group with the higher approval rate (female accounts) also has the higher false-negative rate, missing 10.3% of accounts that go on to default, against 8.9% for male accounts, so the group favored by the headline approval rate is the group whose default risk this threshold captures less completely. Neither comparison is "the" fairness result; both are properties of the same model and threshold, and, as Chapter 18's impossibility result predicts, no adjustment to this one threshold can bring both gaps to zero unless the groups' underlying default rates coincide.

Leaving the protected attributes out of the model mattered: the same XGBoost configuration refit with `SEX`, `MARRIAGE`, and `AGE` as inputs, which the companion notebook runs for contrast, approves 28.6% of male and 33.9% of female accounts at the same threshold, an adverse impact ratio of only 0.843, while scoring essentially no better (test ROC-AUC 0.779 against 0.778). But removing the attributes did not remove the difference. The model never sees `SEX`, yet its decisions still differ measurably by sex, the pattern of Chapter 18's Apple Card vignette: "fairness through unawareness" narrows a disparity only as far as the remaining features fail to encode the omitted attribute. What the audit does not establish is why the residual difference exists: whether it reflects the groups' different default rates or another feature correlated with sex closely enough to reproduce the pattern. Income proxies and credit-limit history are the leading suspects in this dataset, though testing that rigorously would require the counterfactual-fairness framing of Chapter 18's advanced topics, not merely this correlational check. Section 21.4.4 then shows that this single-attribute finding, reassuring as its 0.960 ratio looks, does not remotely tell the whole story once a second characteristic is added.

21.4 From Model to Project

A model is not a project; this part covers deployment, and the template for doing all of it again.

21.4.1 From Model to Deployment: Governance Considerations

Chapter 18's governance section organized model governance around SR 11-7's three pillars, conceptual soundness, ongoing monitoring, and outcomes analysis, which SR 26-2, the guidance that superseded it in April 2026, keeps intact (Federal Reserve, OCC, and FDIC 2026), and this pipeline maps onto all three.

Conceptual soundness is Section 21.3.1's model bake-off and Section 21.3.2's calibration check, documented well enough that an independent validator, Chapter 15's second line of defense, could reproduce every number in Table 21.2 from the companion notebook without re-deriving the modeling choices. Outcomes analysis is Section 21.3.6's fairness audit, which this chapter's numbers show passing on one cut (0.960 by sex) and failing on another (Section 21.4.4's sex-by-age cut), so a one-time check at deployment is not enough; ongoing

Wulin.Suo@Queensu.ca

monitoring, with Chapter 18's population stability index on this feature set and a recurring rerun of both audits, is what would catch a shift in the applicant pool moving either ratio before an examiner does.

None of this replaces human judgment: Section 21.3.5's cost assumptions, the $500-to-$50 ratio driving the threshold, are a business decision no algorithm here can make for the institution, the kind of accountable, documented judgment that Chapter 15's three-lines-of-defense framework and the NIST AI RMF's "govern" function exist to make explicit.

A deployment checklist follows from the chapter's sections. Before the model scores a real applicant, an institution would need Section 21.2.1's cleaning logic and Section 21.2.2's feature engineering encoded in a reproducible data pipeline rather than a one-off notebook cell. It would need Section 21.3.1's model selection and Section 21.4.3's hyperparameter search documented well enough for a validator to rerun them and obtain the same numbers. It would need Section 21.3.2's calibration check and Section 21.3.5's cost assumptions re-approved on a recurring basis, since calibration and an institution's true cost ratio both drift as the applicant population and business environment change. And it would need the fairness audits of Sections 21.3.6 and 21.4.4, the intersectional cut included, rerun on every new cohort of applicants, because Section 21.4.4 showed that a single-attribute check passing today is no guarantee that a deeper cut will not fail tomorrow.

21.4.2 Advanced Topics

Two further analyses go beyond a first pass: a systematic search over XGBoost's hyperparameters, to see how much Section 21.3.1's single configuration left on the table, and a fairness audit that splits the test set two ways at once, revealing a disparity Section 21.3.6's single-attribute audit could not see.

21.4.3 Hyperparameter Tuning: Diminishing Returns in Practice

Section 21.3.1's XGBoost model used a single, manually chosen configuration (300 trees, maximum depth 4, learning rate 0.05). A randomized search over 30 candidate configurations varied tree depth (3 to 6), learning rate (0.01 to 0.1), the number of boosting rounds (100 to 500), the row and column subsampling fractions, and the minimum child weight controlling how conservatively a tree can split. Every candidate was evaluated by five-fold cross-validation on the training set alone, never touching the test set: Section 21.2.2's train/test discipline applied one level deeper, since a hyperparameter search must also be validated on data it never saw.

The best configuration found, depth 4, learning rate 0.1, 100 rounds, no row subsampling, 60% column subsampling, and a minimum child weight of 5, achieved a cross-validated ROC-AUC of 0.786, and, refit on the full training set and evaluated once on the held-out test set, a test ROC-AUC of 0.781, a PR-AUC of 0.554, and a Brier score of 0.135. Compared against Section 21.3.1's untuned XGBoost model (test ROC-AUC 0.778, PR-AUC 0.557, Brier 0.135), the result of a reasonably thorough search is not even a uniform improvement: ROC-AUC rises by three thousandths, PR-AUC falls by three thousandths, and the Brier score does not move at the third decimal, all changes far smaller than the gap either tree ensemble opened up over logistic regression in Table 21.2.

This is a useful, if unexciting, finding. Once a model family is well chosen and its configuration set to sensible, moderate values, as Section 21.3.1's untuned model already was, further search on a dataset of this size and feature set faces diminishing returns. The gap between the cross-validated score (0.786) and the held-out test score (0.781) is a small reminder that a search's selected score is mildly optimistic, having been chosen among 30 candidates because it performed best on that particular split.

21.4.4 Intersectional Fairness: What Aggregation Can Hide

Section 21.3.6's audit split the test set only by sex. Splitting it a second way at the same time, by sex and by an age cutoff at 30 years, reveals a pattern the single-attribute audit could not, shown in Table 21.7.

The most disadvantaged subgroup by approval rate, young men under 30, is approved only 23.9% of the time, against 33.5% for the most favored subgroup, men 30 and over. That is an adverse impact ratio, from the unrounded rates, of 23.92%/33.53% = 0.713, a clear four-fifths-rule violation, and it stands in sharp contrast to Section 21.3.6's sex-only audit, which found a ratio of 0.960, comfortably on the right side of the same threshold. The two findings are not in tension; they answer different questions. Averaging young and

Wulin.Suo@Queensu.ca

Sex	Age group	n	Approval rate	False-negative rate
Male	Under 30	669	23.9%	4.7%
Male	30 and over	1,733	33.5%	10.5%
Female	Under 30	1,243	30.0%	10.1%
Female	30 and over	2,355	33.3%	10.4%

Table 21.7: An intersectional fairness cut: sex crossed with age group

older men together in the sex-only audit hides that young men, on their own, face a starkly lower approval rate than any other subgroup, a disparity the single-attribute view cannot see because it never asks. Nor is this simply an age effect that a separate age audit would have caught at the same strength: among women the under-30 group is approved at 30.0% against 33.3%, a ratio of about 0.90, so the shortfall is concentrated where the two characteristics meet. It also arises in a model that, as Section 21.2.2 specified, never sees `AGE` at all, a second and sharper instance of Section 21.3.6's point that leaving an attribute out does not keep the features that correlate with it, shorter credit histories and lower assigned limits among the youngest borrowers, from carrying it back in; the most obvious such feature here is `LIMIT_BAL`, since the youngest borrowers, and young men above all, hold markedly lower credit limits (a median of NT$80,000 for men under 30 in the test set, against NT$150,000 for men 30 and over). This is the failure mode intersectional or subgroup analysis exists to catch, and that fair-lending examiners and model validators test for: a model can pass every single-attribute audit an institution runs and still produce a stark disparity for a subgroup defined by two or more characteristics, and the only way to find it is to look for it, since Table 21.6's aggregate numbers give no hint that it exists.

A thorough fairness audit therefore cannot stop at a single-attribute cut, however clean its result; it must cross every combination of characteristics an institution has reason to worry about, a much larger testing burden than a single audit table suggests, and, as this dataset shows, not merely a theoretical one. The 2026 change in U.S. federal law does not remove that burden. Since 21 July 2026 Regulation B no longer carries a disparate-impact "effects test" under the Equal Credit Opportunity Act (CFPB 2026), so a ratio below 0.80 is not by itself a federal fair-lending violation; but the four-fifths rule remains the standard screening heuristic, state enforcers and regimes abroad still reach disparate outcomes (Chapter 18's table of regulatory regimes), and model-risk governance expects such a disparity to be found and explained either way.

21.4.5 A Template for Completing a Capstone Project

Section 21.4.6 lists several real datasets a reader could build a capstone project on. This section is the template for doing so: a stage-by-stage checklist distilled from this chapter's pipeline (Figure 21.1), so that a reader starting from any dataset on that list, or one of their own, has a concrete standard to work toward rather than an open-ended instruction to "build a model." Each stage in Table 21.8 names the tool this chapter or an earlier one developed for it, along with a deliverable, a specific artifact, table, or number, that the stage must produce before the project moves on. A project that cannot produce every deliverable is not finished, whatever its headline accuracy.

Stage 9: Deploy and Monitor

The first eight stages end with a model that is correct in a notebook. Stage 9 asks whether it stays correct once it runs unattended, and it is the stage a capstone project is most tempted to describe rather than demonstrate. Section 21.4.1's checklist said what an institution would need before the model scores a real applicant; Stage 9 turns that list into four checks, each one an earlier stage's discipline restated as a property of a running system (Sculley et al. 2015).

Feature parity: training-serving skew. Stage 2's leakage rule, use only information available at decision time, is enforced in a notebook by the analyst's care and in production by the plumbing, and the two can quietly disagree. A data warehouse that already holds the decision day computes a "three-day average balance" over days $t-2$ to t; a live service, which cannot see day t yet, computes it over $t-3$ to $t-1$. For a customer whose daily balances are $1,000, $1,000, $1,000 and then $200 on the decision day, the

Wulin.Suo@Queensu.ca

Stage	What this stage requires	Deliverable to produce and report
1. Problem framing and dataset selection	State the business decision the model will inform in one sentence (approve or deny a loan, flag or clear a transaction, and so on); confirm the chosen dataset has a genuine, non-leaking target label and enough rows to support a genuinely held-out test set.	A one-paragraph problem statement naming the target variable, its base rate, and the decision the model's output will feed into.
2. Data cleaning and feature engineering	Inspect every column's summary statistics and value counts before fitting anything, exactly Section 21.2.1's method; document any undocumented or anomalous codes found and how each was resolved; engineer at least one derived feature not present in the raw columns, and explicitly check it (and every other feature) for information that would not genuinely have been available at the moment a real prediction would be made, Section 21.2.2's leakage discipline.	A short data-quality memo listing every anomaly found and the cleaning decision made, plus the engineered feature list with a one-sentence leakage justification for each.
3. Train/test discipline and model bake-off	A stratified train/test split with a fixed, reported random seed; at least one intrinsically interpretable model (linear or logistic regression) and at least one tree ensemble, evaluated once, on the held-out test set alone, on accuracy, precision, recall, F_1, ROC-AUC, PR-AUC, and Brier score, Section 21.3.1's method.	A results table in the same format as Table 21.2, plus both sets' target base rates confirming the stratification held.
4. Calibration check	A reliability table (deciles of predicted probability against actual outcome rate) for at least the best-discriminating model, Section 21.3.2's method.	A calibration table in the format of Table 21.3, plus one sentence stating whether the model's raw probability, not merely its ranking, can be trusted at face value.
5. Explainability	A global importance ranking (permutation importance or mean absolute SHAP value) and at least one individual, local explanation for a specific prediction, Section 21.3.4's method.	A top-five global feature ranking and one specific case's local explanation, with its completeness check (SHAP values plus base value reproducing the model's actual output) shown explicitly.
6. Threshold and cost analysis	An explicit, stated assumption for the dollar cost of a false negative and a false positive, and the resulting cost-minimizing threshold found by sweeping a grid of candidate thresholds, Section 21.3.5's method.	The two assumed costs, the resulting threshold, and its total expected cost compared explicitly against the naive 0.5 threshold's cost.
7. Fairness audit	At least one single-attribute fairness cut (approval rate, the four-fifths adverse impact ratio, and a false-negative/false-positive rate comparison) and, wherever the data contains a second relevant characteristic, one intersectional cut crossing two attributes at once, Sections 21.3.6 and 21.4.4's method.	An approval-rate table and adverse impact ratio for at least one single-attribute cut, and, if attempted, one intersectional cut, with an explicit statement of whether either violates the four-fifths rule.
8. Governance write-up	A short memo addressing the three SR 11-7 pillars (now carried by SR 26-2) for this specific model: what conceptual soundness was checked, what ongoing monitoring (a population stability index or a re-audit cadence) is proposed, and what outcomes analysis would be needed before this model could be trusted with a real decision, Section 21.4.1's method.	A half-page governance memo naming a concrete monitoring metric, its planned check frequency, and who (which of Chapter 15's three lines of defense) would be responsible for acting on it.
9. Deploy and monitor	Show that the model computed in production is the model that was validated: one shared definition of every feature, versioned artifacts and lineage that let any past decision be rebuilt, a staged release with a rollback trigger fixed in advance, and monitoring whose alarms start a governed retraining rather than an automatic one.	A deployment plan naming the feature-parity check, the artifact and lineage record, the release stages and rollback trigger, and the monitored metrics with the values that trigger review and retraining.

Table 21.8: A nine-stage template for a capstone project, distilled from this chapter's own pipeline

Wulin.Suo@Queensu.ca

two definitions give $733.33 and $1,000.00, and a one-feature scorecard PD $= 1/(1 + e^{-(1.2-0.0025x)})$ turns them into 34.7% and 21.4%. Against a 30% cutoff the model was validated to decline this applicant and production approves them, understating the probability of default by 13.3 points, 38% of the correct figure, with no error raised anywhere. The deliverable is a parity test: score a sample of recent applicants through the training code and through the serving path, and require identical feature values. A feature store that holds one definition of each feature, computed once and served to both, is the standard structural answer.

Lineage: the rebuild test. Stage 5's explanation must be regenerable months later, since an adverse-action notice explains the decision the model made then, not the one it would make now. That requires versioning the whole artifact, preprocessing, fitted parameters, feature definitions, and Stage 6's threshold, together with the identity of its training data and the library versions, so that any past score can be rebuilt exactly. The deliverable is a rebuild of one past decision from stored records alone.

Release: shadow, challenger, rollback. A validated model is a hypothesis about production. Run it first in shadow, scoring live applications while its outputs affect nothing, which is also the cheapest way to catch skew; then let it decide a defined slice of applications as a challenger to the incumbent; then widen the slice in stages. The rollback trigger, a named metric crossing a named value, is fixed before the release, because a trigger chosen while a model is misbehaving tends to be chosen to justify keeping it, and the incumbent stays runnable throughout.

Monitoring and retraining triggers. Monitor the inputs and score distribution with Chapter 18's population stability index, using its conventional bands, below 0.10 no material change, 0.10 to 0.25 worth investigating, above 0.25 requiring revalidation, and monitor calibration on realized defaults as they mature, since calibration usually decays first, together with Stage 7's approval-rate ratios on each new cohort. An alarm should start a retraining run whose output re-enters the release sequence as a challenger, with Stages 4 through 7 repeated before promotion; it should not trigger an automatic refit. Automatic retraining turns a governed model change into an unreviewed one, and in lending it learns from outcomes observed only for applicants the incumbent approved, a feedback loop that can entrench the incumbent's errors rather than correct them.

A project report built on this template falls into five sections, usable directly as a report or presentation outline. The **executive summary** states Stage 1's problem statement and the headline result in one paragraph a non-technical reader could act on. **Data and methodology** covers Stages 2 and 3, written so that another analyst could reproduce the cleaning, feature engineering, and model choices exactly. **Results** presents Stages 3 and 4's tables together, so a reader can judge discrimination and calibration side by side rather than seeing only the most favorable number. **Explainability and fairness** covers Stages 5 through 7, which belong together: an explanation of what drives the model's decisions is also evidence for or against a fairness concern, as Section 21.3.6's discussion of `LIMIT_BAL` and Section 21.4.4's intersectional finding showed. **Limitations and recommendations** carries Stage 8's governance memo and Stage 9's deployment plan, with an explicit, named list of what the project did not check.

That last list is the one most often skipped, and it matters most. Section 21.4.4 showed that an unchecked intersectional cut can hide a violation the report's headline numbers give no hint of. A capstone report that does not name its blind spots is not more trustworthy for omitting them; it is less honest about a limitation every model in this chapter shares.

Two failure modes are the most common ways a project satisfies this template's letter while missing its point. The first is treating each stage as an isolated checkbox rather than letting later stages inform earlier ones: Section 21.3.2's finding that the plain logistic regression's safest decile was miscalibrated should change which model Stages 6 and 7 are run on, not be reported and then ignored in favor of the model with the best Stage 3 accuracy. The second is stopping the fairness audit (Stage 7) at the first cut that passes, the mistake this chapter would have made had Section 21.4.4 never been run; a single-attribute cut that passes cleanly is a reason to look harder for an intersectional disparity, not to stop looking.

21.4.6 Capstone Project Ideas

The pipeline built in this chapter generalizes to other public financial datasets: clean, engineer features, split, bake off models, calibrate, explain, choose a threshold, audit for fairness. Each of the following is a capstone project on a real, freely available dataset, several of them also distributed on Kaggle under the names given.

Wulin.Suo@Queensu.ca

- **Credit card fraud detection.** The Kaggle/Worldline/ULB "Credit Card Fraud Detection" dataset (284,807 European card transactions, 492 confirmed frauds, features anonymized via PCA) is far more imbalanced than this chapter's 22% default rate, a severe 0.17% positive rate that makes Chapter 15's precision-recall framing, and this chapter's calibration and cost-sensitive threshold methods, much more consequential than here; a reader completing this project should expect the naive 0.5 threshold to be almost useless, and should find that Section 21.3.5's analytic rule $c_{FP}/(c_{FP} + c_{FN})$ still locates the cost-minimizing threshold only if the model's probabilities are calibrated, which is far harder to achieve and to check with 492 positives than with this chapter's 6,636.

- **Consumer lending at greater scale.** The Lending Club loan dataset (millions of originated peer-to-peer loans with real interest rates, loan grades, and repayment outcomes) extends this chapter's pipeline to a dataset large enough that model training time itself becomes a design constraint, and includes enough temporal history to test a model's stability across a full economic cycle, the "non-stationarity" concern Chapter 6 and Chapter 17 both raised only in the abstract.

- **An algorithmic trading backtest on real prices.** Historical daily prices for the S&P 500 constituents (freely available from sources such as Yahoo Finance or Stooq) let a reader rebuild Chapter 13's pairs-trading or Chapter 12's mean-variance framework on real, not fictional, return series, confronting transaction costs, survivorship bias (delisted constituents silently missing from many free datasets), and the gap between an in-sample backtest and true out-of-sample performance.

- **Financial sentiment at scale.** The Financial PhraseBank dataset (thousands of analyst-labeled sentences from financial news, by agreement level) or a Twitter financial-sentiment dataset let a reader rebuild Chapter 16's Loughran-McDonald scoring and Naive Bayes classifier on labeled data, and compare a hand-built lexicon with a fine-tuned transformer, which Chapter 16 only described.

- **A second, independent fraud dataset for cross-dataset generalization.** The IEEE-CIS Fraud Detection dataset (a Kaggle competition dataset with several hundred anonymized features spanning transaction, identity, and device information) is large and high-dimensional enough to make Section 21.3.4's SHAP-based explanation necessary rather than illustrative, since no analyst could manually inspect several hundred features' relationships to a prediction the way this chapter's 25-feature model still just barely permits.

- **A robo-advisor: automated, risk-based portfolio construction.** Using free historical price data for a diversified set of low-cost exchange-traded funds (the kind of instrument Chapter 3 introduced), a reader can rebuild the core of a robo-advisor, following Chapter 12's own worked treatment directly: a short risk-tolerance questionnaire mapped to a target asset allocation, the mean-variance optimizer (or its risk-parity variant) computing the actual portfolio weights on real rather than illustrative return data, periodic rebalancing back to those target weights as prices drift, and, for a realistic version, tax-loss harvesting logic and a fee-drag comparison against a comparable actively managed alternative, extending Chapter 12's stylized numbers to a real portfolio and, ideally, a second real case besides Schwab's to compare against. This project differs from every other one in this list in an instructive way: it is not a classification or regression problem at all, but an optimization and decision-rule problem, so Chapter 18's SHAP and permutation-importance tools do not directly apply, and the natural "explainability" question becomes instead whether the resulting portfolio weights are traceable back to the questionnaire's stated risk tolerance and Chapter 12's stated optimization objective, a different, allocation-based notion of interpretability worth contrasting explicitly with this chapter's classification-based one.

- **Derivative pricing and delta-hedging on real options data.** Historical options-chain data for a liquid underlying such as the S&P 500 ETF (available from sources including Yahoo Finance, CBOE historical data files, and options-focused Kaggle datasets) lets a reader confront Chapter 5's Black-Scholes-Merton and binomial models with actual market prices rather than an assumed volatility input: back out each option's implied volatility from its traded price, plot the resulting volatility smile or skew across strikes (a pattern Chapter 5's constant-volatility assumption cannot produce at all, making its own limitation directly visible), and then implement and backtest a discrete-time delta-hedging

Wulin.Suo@Queensu.ca

strategy, rebalancing a hedge position at fixed intervals and tracking the resulting hedging error against the theoretical, continuous-rebalancing ideal Chapter 5 derives. This project is a natural pair with Chapter 14's market-microstructure material, since a real hedging desk's rebalancing frequency is itself constrained by the transaction costs Chapter 14 modeled, so a reader can extend the backtest to show how hedging error and transaction cost trade off against each other as the rebalancing interval shrinks.

- **Portfolio risk management through a real historical crisis.** Real daily return data for a multi-asset portfolio (a mix of equity, bond, and commodity ETFs, again freely available from sources such as Yahoo Finance or Stooq) lets a reader rebuild Chapter 8's Value-at-Risk and Expected Shortfall machinery, parametric, historical-simulation, and Monte Carlo, on real returns rather than a small illustrative covariance matrix, and then backtest the resulting VaR estimates using Chapter 8's own exception-counting method across a window that spans a real historical stress period, the 2008 financial crisis or the 2020 pandemic crash, both of which are included in most free multi-year daily price histories. The point of choosing a window with a real crisis in it deliberately is to test the assumption every one of Chapter 8's VaR methods quietly makes, that recent historical volatility and correlation are a reasonable guide to near-term risk, against a period where that assumption visibly failed, correlations across asset classes that looked low in calm markets rose sharply during the crisis itself, a phenomenon a reader can verify directly by comparing the portfolio's rolling covariance matrix estimated inside versus outside the crisis window, before feeding either estimate into Chapter 12's mean-variance or risk-parity optimizer and comparing the resulting portfolios' actual, realized risk during the crisis itself.
- **A smaller-scale replication for a first project.** The Statlog German Credit dataset (1,000 applicants, 20 features), also mirrored on Kaggle as "German Credit Risk," is small enough to rerun this entire chapter's pipeline, cleaning, feature engineering, bake-off, calibration, SHAP, threshold choice, fairness audit, in an afternoon, making it a natural first capstone for a reader who wants to practice the full workflow once, end to end, before tackling one of the larger datasets above.

21.5 Discussion Questions

1. Table 21.2 shows the class-weighted logistic regression achieving the best F_1 score among the two logistic models but the worst Brier score of all four models. If you were advising a lender that needed to report a single default probability to a regulator for every applicant, which of the four models would you recommend, and would your answer change if the lender only needed to rank applicants from safest to riskiest rather than report a specific probability to anyone?

2. Section 21.2.1 cleaned two undocumented category codes by folding them into an existing "other" category. Discuss an alternative cleaning strategy (for instance, treating each undocumented code as missing data and imputing it, or as its own separate category) and what evidence in the data would help you decide which approach is more defensible.

3. Section 21.3.4 showed permutation importance and SHAP disagreeing on whether `PAY_1` or `max_delinquency` is more important. If a regulator asked your institution which single feature drives most of your credit model's decisions, how would you respond given this disagreement, and does the question itself presuppose something that may not be true of the model?

4. Section 21.3.5's cost-minimizing threshold depends entirely on the assumed \$500-to-\$50 cost ratio. Propose a concrete way an institution could estimate these two costs from its own historical data, and discuss whether the same cost ratio should apply uniformly to every applicant or could reasonably vary by loan size.

5. Section 21.3.6's adverse impact ratio (0.960) passes the four-fifths rule comfortably, yet the same model refit with `SEX`, `MARRIAGE`, and `AGE` as inputs scored 0.843, and Section 21.4.4's sex-by-age cut fails (0.713). Discuss what additional analysis, beyond recomputing these same ratios next quarter, an internal model risk function should perform to decide whether the passing result reflects a stable, defensible model, and how much of the remaining disparity it should attribute to proxies for the omitted attributes rather than to genuine differences in default risk.

Wulin.Suo@Queensu.ca

6. Choose one of Section 21.4.6's capstone project ideas and walk through Table 21.8's nine stages for it explicitly, one sentence per stage naming the specific dataset column, model choice, or metric that stage would use, and flag any stage whose deliverable that project's dataset cannot actually support (for instance, a dataset with no protected-characteristic field at all, which would leave Stage 7 unable to produce a real fairness cut).

7. Section 21.4.6's robo-advisor and portfolio-risk-management projects are not classification problems and have no natural confusion matrix, ROC curve, or SHAP value. For either one, propose a concrete way to check whether the resulting system is behaving as intended, given that this chapter's usual evaluation toolkit does not directly apply, and identify which of Table 21.8's nine stages would need to be reinterpreted rather than dropped entirely.

8. Section 21.4.4's intersectional cut checked sex crossed with age. Propose a third characteristic from this dataset worth crossing with sex in a real fair-lending examination, justify your choice, and discuss what evidence would tell you whether it is worth checking before you actually run the numbers.

9. Section 21.4.5 names two failure modes: treating each template stage as an isolated checkbox, and stopping a fairness audit at the first cut that happens to pass. Describe a plausible way a well-intentioned project team could fall into each failure mode without realizing it, and propose one concrete process change (not a new statistical method) that would catch each one before a project is reported as finished.

21.6 Challenges in End-to-End Financial AI Projects

Several challenges recur across this chapter's pipeline and connect directly to themes from earlier chapters.

Real data is never clean by default. Section 21.2.1's undocumented category codes have no counterpart in any earlier chapter's fictional dataset, and a project that skips this step inherits whatever silent errors those codes introduce into every downstream model.

The best model by one metric is not always the best by every metric. Table 21.2's class-weighted logistic regression has the best F_1 and the worst Brier score of the four models, so "best model" is well defined only once the deployment use case fixes which metric matters.

Different explanation methods can disagree, and both can be right. Section 21.3.4's permutation-importance-versus-SHAP disagreement on the top two features is not a bug in either method but a consequence of the two answering different questions, the caution Chapter 18 raised and this chapter confirms empirically.

A threshold choice is a business decision with real financial consequences, not a modeling detail. Section 21.3.5's more than twofold cost difference between the naive and cost-optimal thresholds dwarfs the gap between any two models in the bake-off, a quantified instance of Chapter 15's alert-economics argument.

Removing a protected attribute is not the same as removing its effect. Leaving `SEX`, `MARRIAGE`, and `AGE` out of the model, as fair-lending law requires, raised Section 21.3.6's sex-only adverse impact ratio from 0.843 to 0.960 at almost no cost in accuracy, yet the model still treats the sexes measurably differently and, never having seen `AGE`, still approves young men at well under four-fifths of the rate of older men, which is why Section 21.4.1 treated ongoing outcome monitoring, not the choice of inputs alone, as the real governance obligation.

A single-attribute fairness audit can pass while an intersectional one fails. Section 21.4.4's sex-and-age cut found a clear four-fifths-rule violation (0.713) that Section 21.3.6's sex-only audit (0.960, passing) gave no hint of, so an institution that stops testing after the first clean-looking result may never discover a disparity that exists for a subgroup it never checked.

Not every financial AI project is a classification problem, and explainability means something different for each. Section 21.4.6's robo-advisor project has no confusion matrix, ROC curve, or natural SHAP value, since it is an optimization problem rather than a supervised-learning one, so "explaining" its output means tracing portfolio weights back to a stated risk tolerance and optimization objective, a different task from those Section 21.3.4's methods were built for, and a reminder that this book's explainability toolkit, powerful as it is for classification and regression, is not a universal solvent.

An end-to-end pipeline is only as trustworthy as its reproducibility. Every number in this chapter came from a fixed random seed and a documented sequence of cleaning, feature engineering, and modeling

Wulin.Suo@Queensu.ca

choices, recorded in the companion notebook so that Section 21.4.1's independent-validation obligation can actually be met.

21.7 Key Takeaways

The chapter's dataset, the UCI/Kaggle Default of Credit Card Clients data, was the book's first not built to illustrate anything, and it surfaced problems the fictional datasets never could: undocumented category and status codes and a misnamed column to clean (Section 21.2.1), a class imbalance whose remedy (class weighting) traded a better F_1 score for a worse Brier score (Section 21.3.1), and a calibration gap in the plain logistic regression's safest-looking decile that a rank-based evaluation would never have surfaced (Section 21.3.2).

Running Chapter 18's explanation toolkit with real software, `shap`'s `TreeExplainer` and scikit-learn's permutation importance, on a nonlinear gradient-boosted model confirmed the completeness property Chapter 18 verified by hand and exposed a disagreement between two legitimate importance rankings (Section 21.3.4).

A cost-sensitive threshold cut expected cost by more than half relative to the naive 0.5 default (Section 21.3.5), though at an operating point far stricter than any real lender would accept, an artifact of the toy cost ratio that the analytic threshold $c_{FP}/(c_{FP} + c_{FN})$ makes transparent. The model, built without `SEX`, `MARRIAGE`, or `AGE`, passed a sex-only fairness audit comfortably but still showed a residual sex difference and the approval-rate-versus-error-rate tension Chapter 18 demonstrated with synthetic data. Leaving out a protected attribute is not fairness, and a single deployment-time check is not governance; ongoing monitoring is (Section 21.4.1).

Section 21.4.2 went deeper in two directions. A systematic hyperparameter search found no consistent gain over Section 21.3.1's single XGBoost configuration, a reminder that model-family choice and feature engineering matter far more than fine-tuning once a sensible configuration is in place. An intersectional cut by sex and age then uncovered a stark, four-fifths-rule-violating disparity for young male applicants that Section 21.3.6's single-attribute audit, run first and reported as passing, gave no hint of: the book's sharpest illustration of why an audit's scope, not just its arithmetic, determines what it can find.

The breadth this one pipeline drew on is the point of a capstone chapter. Chapter 2's pandas and NumPy fundamentals ran every line of the code. Chapter 6's train/test discipline, feature engineering, bagging and boosting, and overfitting caution structured Sections 21.2.2 and 21.3.1. Chapter 9's calibration caveat became Section 21.3.2's central finding. Chapter 15's class-imbalance remedies and cost-sensitive threshold framing became the empirical results of Sections 21.3.1 and 21.3.5. And Chapter 18's explainability and fairness toolkit, moved from hand computation on a linear model to real software on an ensemble, became Sections 21.3.4, 21.3.6, and 21.4.4.

A single real dataset, messier than any this book constructed, was enough to put nearly every tool the book developed to simultaneous use. None of these methods was meant to be practiced in isolation, and a real financial AI project rarely allows using only one at a time.

Section 21.4.5 distilled the pipeline into a nine-stage template: problem framing, data cleaning and feature engineering, model bake-off, calibration, explainability, threshold and cost analysis, fairness audit, a governance write-up, and a deployment and monitoring plan that keeps the validated model the one actually running. Each stage names its required deliverable, so that a reader knows what "finished" means.

The chapter, and the book, close with Section 21.4.6's further project ideas, each a real, freely available dataset, spanning fraud detection, consumer lending, algorithmic trading, financial sentiment, robo-advisory, derivatives hedging, and credit scoring at a different scale. They are for a reader now ready to run the same disciplined pipeline independently on data this book did not hand-pick: clean, engineer, split, compare, calibrate, explain, price the threshold, and audit for fairness.

21.8 Suggested Exercises

21.1 Using Section 21.2.1's method, tabulate the default rate separately for accounts with an undocumented `EDUCATION` code (0, 5, or 6) before cleaning, and compare it to the default rate for documented codes

Wulin.Suo@Queensu.ca

1 through 4, discussing whether folding the undocumented codes into category 4 seems reasonable given what you find.

21.2 Using Table 21.2, compute each model's precision-recall trade-off directly (precision divided by recall) and rank the four models by this ratio, then explain what a very high or very low ratio implies about a model's operating point.

21.3 Using Section 21.3.2's method, compute the calibration gap (mean predicted probability minus mean actual default rate) for every decile of the random forest's predictions, and state whether the random forest's calibration more closely resembles the plain logistic regression's or XGBoost's in Table 21.3.

21.4 Section 21.2.2 kept `AGE` out of the model. Refit the XGBoost model with `AGE` added back as a 26th feature, use Section 21.3.4's method to compute its mean absolute SHAP value and its rank among the 26 features, and rerun Table 21.7's audit. Discuss, in light of Chapter 18's discussion of proxy features correlated with protected characteristics, whether leaving `AGE` out actually removed age-related information from the model's decisions.

21.5 Using Section 21.3.5's method, recompute the cost-minimizing threshold if the cost of a missed default is revised from \$500 to \$1,000 (holding the \$50 false-positive cost fixed), and explain in words the direction you expect the optimal threshold to move before recomputing it.

21.6 Using Section 21.3.6's method, recompute the adverse impact ratio and the false-negative-rate gap at the naive 0.5 threshold instead of the cost-minimizing 0.10 threshold, and discuss whether the choice of threshold changes which group appears more favored by each of the two statistics.

21.7 Using Section 21.4.6's description of the Kaggle credit card fraud dataset, explain why a 0.17% positive rate makes accuracy an even less useful headline metric than it was for this chapter's 22.1% default rate, referencing Chapter 15's original caution about accuracy under class imbalance.

21.8 Using Table 21.7's method, compute the adverse impact ratio for a third cut, sex crossed with `EDUCATION` (collapsed to graduate school versus all other categories), and state whether this cut reveals a four-fifths-rule violation as stark as the sex-and-age cut in the text.

21.9 Section 21.4.3 found no consistent improvement from hyperparameter tuning. Propose one plausible reason a much larger or higher-dimensional dataset (such as the IEEE-CIS fraud dataset in Section 21.4.6) might show a larger gain from the same kind of search, referencing the bias-variance discussion in Chapter 6.

21.10 Using Section 21.3.3's isotonic recalibration result, explain why the recalibrated logistic regression's ROC-AUC barely changed (0.720 to 0.724) even though its Brier score improved measurably, referencing the monotonicity property isotonic regression is constrained to satisfy.

21.11 Explain why fitting an isotonic recalibration function directly on the test set, rather than via cross-validation on the training set alone, would invalidate the resulting calibration check, referencing Section 21.2.2's train/test discipline.

21.12 Using Stage 9's training-serving skew example in Section 21.4.5, with the same balances of \$1,000 on days $t-3$ through $t-1$ and the same scorecard and 30% cutoff: (a) recompute both probabilities of default if the decision-day balance is \$600 rather than \$200, and state whether the two code paths now reach the same decision; (b) find the decision-day balance below which the trained model declines the customer while the serving path still approves; (c) explain why a monitoring dashboard that tracks only the average served PD would not reveal this skew, and propose the test that would.

21.9 Further Reading

- Yeh and Lien (2009), "The Comparisons of Data Mining Techniques for the Predictive Accuracy of Probability of Default of Credit Card Clients." The original paper introducing the dataset this chapter's case study is built on.

Wulin.Suo@Queensu.ca

- UCI Machine Learning Repository (n.d.), *Default of Credit Card Clients Data Set*. The public hosting page for the dataset used throughout this chapter, including its full data dictionary.
- Chen and Guestrin (2016), "XGBoost: A Scalable Tree Boosting System." The original paper describing the gradient-boosting implementation used in Section 21.3.1.
- Lundberg and Lee (2017), "A Unified Approach to Interpreting Model Predictions." The SHAP paper underlying the `TreeExplainer` algorithm used in Section 21.3.4, previously cited in Chapter 18 for its linear-model application.
- Niculescu-Mizil and Caruana (2005), "Predicting Good Probabilities with Supervised Learning." A foundational study of which model families tend to produce well-calibrated probabilities, directly relevant to Section 21.3.2's findings.
- Provost and Fawcett (2013), *Data Science for Business*. A practitioner-oriented treatment of the cost-sensitive threshold reasoning developed in Section 21.3.5.
- Dua and Graff (2019), *UCI Machine Learning Repository*. The broader repository this chapter's dataset is drawn from, and a starting point for several of Section 21.4.6's further project ideas.
- Sculley et al. (2015), "Hidden Technical Debt in Machine Learning Systems." The classic account of why machine-learning systems accumulate maintenance debt in data and configuration rather than code, the concerns behind Stage 9 of Section 21.4.5's template.
- Buolamwini and Gebru (2018), "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification." The landmark empirical demonstration that a system can pass single-attribute fairness checks while still showing a stark disparity for an intersectional subgroup, the same pattern Section 21.4.4 found directly in a lending context.

Wulin.Suo@Queensu.ca